

FitPlanner

Aplicación Web para la Planificación de Rutinas de Ejercicios

Alejandro Lago Pereira

FP Superior DAW 2 · IES Fernando Wirtz

FitPlanner

Aplicación Web para la Planificación de Rutinas de Ejercicios

Nombre: Alejandro Lago Pereira

Centro: Fernando Wirtz

Curso: FP Superior – DAW 2

Indice

Contexto.....	3
Objetivo.....	3
Alcance.....	3
Funcionales.....	4
No Funcionales.....	4
Flujo de Datos.....	6
Login (Login.jsx).....	9
Register (Register.jsx).....	9
Sidebar (Sidebar.jsx).....	9
Página de Rutinas (Routines.jsx).....	10
Página de Planificación (Planning.jsx).....	11
Página de Ejercicios (Exercises.jsx).....	12
Modal de perfil (ProfileModal.jsx).....	13
Tema Claro/Oscuro.....	13
Diseño Responsive.....	13
Configuración de la aplicación (Program).....	14
Autenticación JWT (AuthController).....	14
CRUD de Rutinas (RoutineController).....	15
Gestión del perfil (ProfileController).....	16
Integración con API externa (ExerciseApiService).....	16
Configuración de Entity Framework (AppDbContext).....	17

Introducción

Contexto

Este proyecto surgió a un problema que solía surgir tanto a mi como a mis compañeros de gimnasio tiempo atrás, y era el de planificar al momento que serie de ejercicios queríamos hacer. Varias veces nos quedábamos pensando “Hoy queremos hacer espalda, pero no sabemos que serie ejercicios apuntarían a dicha sección”. De ahí me surgió esta idea. La de poder planificar múltiples rutinas para X días de la semana y poder ajustarlo fácilmente según la necesidad y repetir la rutina o modificarla sin problemas.

Objetivo

Facilitar y mejorar la calidad de ejercicios físicos mediante planificaciones de rutinas a lo largo de la semana, además de poder investigar dichos ejercicios, ver un GIF que muestra su movimiento y una serie de instrucciones y herramientas de ser necesario

Alcance

Uso de Logins y Registro de usuarios, CRUD de rutinas, buscador de ejercicios a una API externa, perfiles de usuarios

Análisis de Requisitos

Funcionales

- Registro e Inicio de Sesión usando Tokens JWT
- Uso de operaciones CRUD para las rutinas
- Añadir ejercicios a rutinas con series, repeticiones y descansos
- Búsqueda de ejercicios por nombre y filtrarlos por partes del cuerpo
- Mostrar ejercicios de manera visual usando GIFs
- Edición de perfiles (Nombre, biografía, fecha de nacimiento, teléfono y foto de perfil)
- Cambio de temas claro/oscuro
- Sidebar con el que acceder a todas las secciones del proyecto

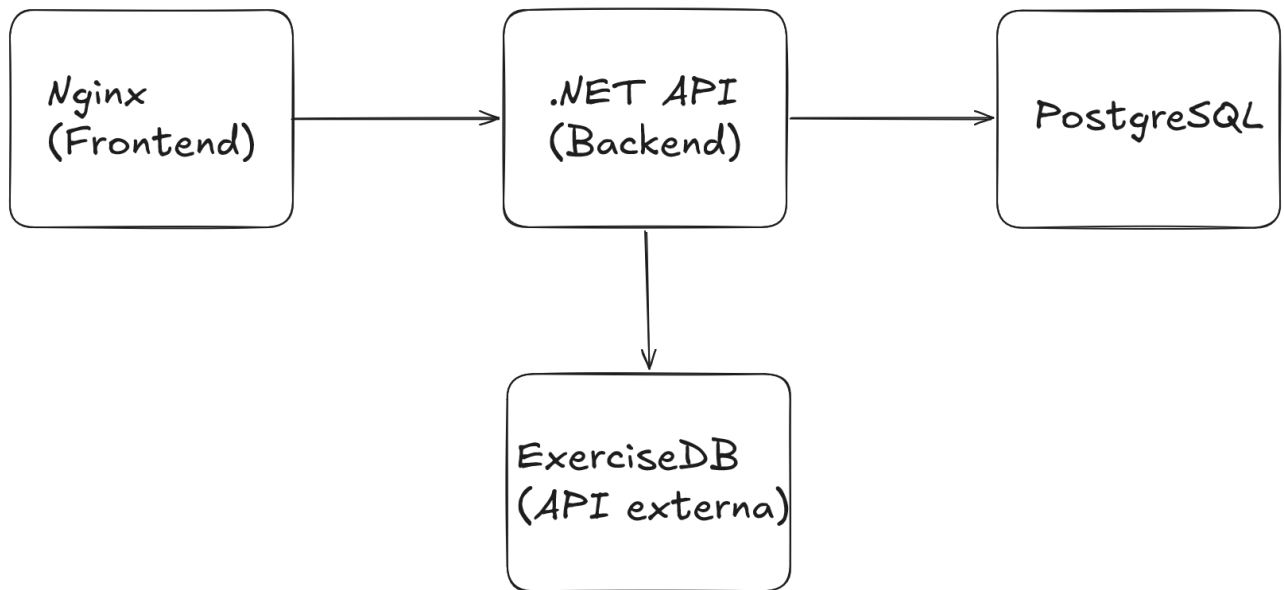
No Funcionales

- Aplicación SPA gracias a REACT y navegación sin recarga
- Uso de API RESTful con ASP.NET Core
- Toda la información almacenada en una base de datos PostgreSQL
- Autenticación segura mediante Tokens JWT y ASP.NET Identity
- Diseño Responsive para todo tipo de dispositivos
- Contenerizado con Docker

Tecnologías Utilizadas

Tecnología	Propósito
C# / .NET	Backend API
ASP.NET Core Identity	Autenticación y Gestión de Usuarios
Entity Framework Core	ORM (Object-Relational Mapping) para base de datos
Npgsql	Driver PostgreSQL
REACT	Frontend SPA
Vite	Bundler y Dev Server
REACT Router	Enrutamiento del frontend
Axios	Cliente HTTP para la API
Lucide REACT	Iconos SVG
PostgreSQL	Base de Datos
Docker	Contenedores
Nginx	Servidor web y Proxy inverso
Cloudflare	DNS y SSL

Arquitectura del Sistema



Flujo de Datos

1. Usuario accede a alejandrolago.com → Cloudflare → Nginx (Frontend)
2. Frontend React carga el SPA
3. Peticiones `/api/*` → Nginx proxy → Backend .NET
4. Backend consulta PostgreSQL o API externa de ejercicios
5. Respuestas JSON → Frontend renderiza

Diseño de la Base de Datos

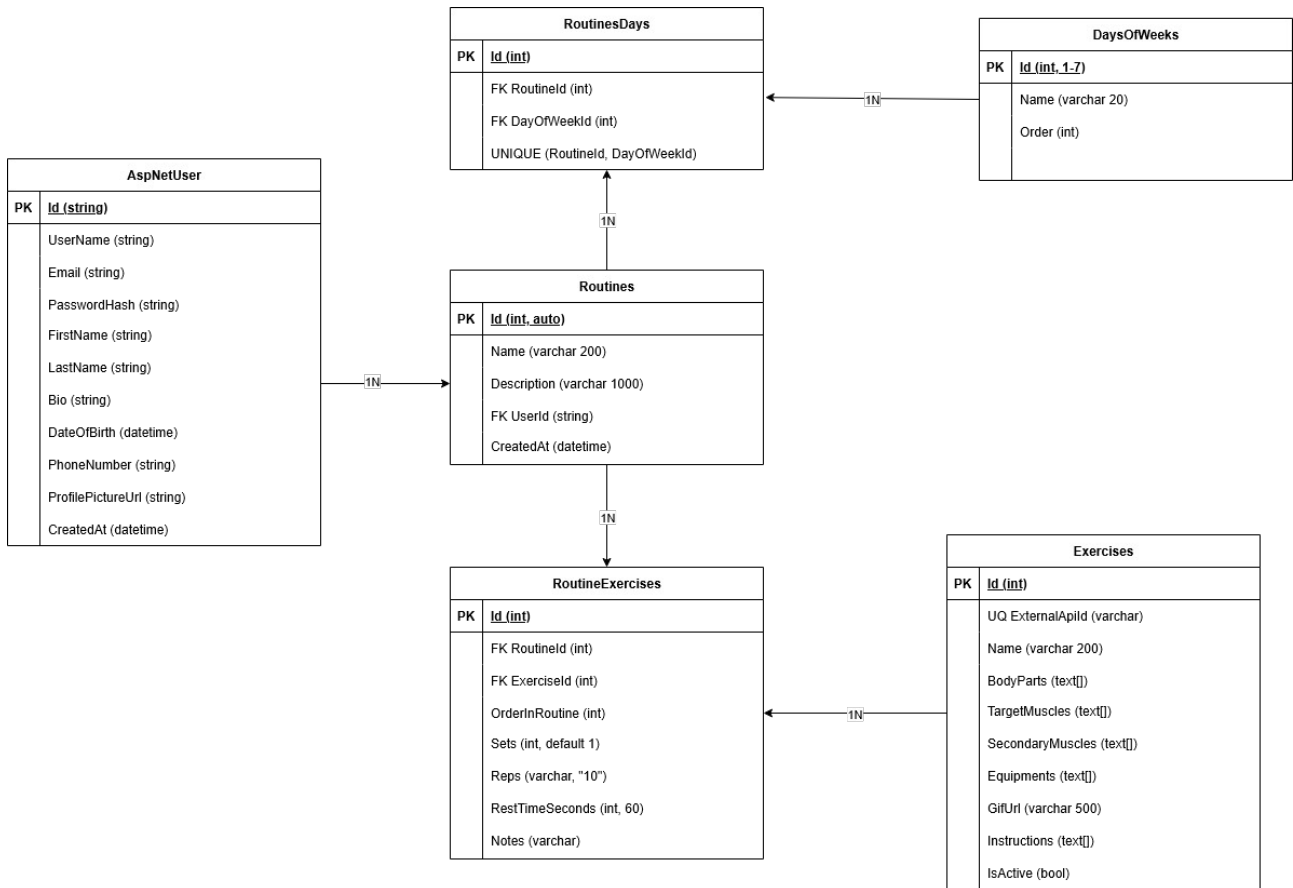


Tabla	Propósito
AspNetUsers	Usuarios
Exercises	Ejercicios
Routines	Rutinas
RoutineDays	Días de la semana asignados a una rutina
RoutineExercises	Ejercicios dentro de una rutina
DaysOfWeek	Días de la semana
AspNetRoles, AspNetUserRoles, etc	Tablas estándar de Identity

Diseño de la API (Endpoints)

Método	Ruta	Autenticación	Descripción
POST	/api/auth/register	No	Registrar usuario
POST	/api/auth/login	No	Iniciar Sesión
GET	/api/profile	Si	Obtener perfil
PUT	/api/profile	Si	Actualizar perfil
GET	/api/routines	Si	Listar rutinas del usuario
POST	/api/routines	Si	Crear rutina
PUT	/api/routines/{}	Si	Actualizar rutina
DELETE	/api/routines/{}	Si	Eliminar rutina
GET	/api/exercise/all	Si	Obtener todos los ejercicios
GET	/api/exercise/search	Si	Buscar ejercicios

Diseño de la Interfaz

Login (Login.jsx)

- Formulario centrado vertical y horizontalmente con tarjeta blanca sobre fondo oscuro
- Campos:
 - Email
 - Password
- Botón “Sign In” con estado de carga
- Enlace “Sign up” para ir al registro
- Logo de FitPlanner (icono Dumbbell) y titulo en la cabecera de la tarjeta
- Mensaje de error mostrado en rojo debajo del formulario si las credenciales son incorrectas
- Validación HTML5 y validación en backend

Register (Register.jsx)

- Misma estructura visual que el login
- Campo adicional : Full Name
- Boton “Sign up”
- Enlace “Sign in” para volver al login
- La contraseña debe cumplir, como mínimo, 6 caracteres, 1 mayuscula y 1 digito

Sidebar (Sidebar.jsx)

- Fija a la izquierda, ancho 260px con un fondo semitransparente
- Logo dumbbell con el texto en naranja
- Navegación con 3 enlaces e iconos
 - Planning(Calendar) → /planning
 - Routines(ListChecks) → /routines
 - Exercises(Search) → /exercises
- Zona inferior
 - Botón para alternar entre tema claro u oscuro
 - Botón “Profile” que abre el modal del perfil
 - Botón “Sign out” que cierra sesión y redirige al login

Página de Rutinas (Routines.jsx)

- Un layout general de dos columnas asimétricas
 - Columna izquierda para listar las tarjetas de rutinas
 - Columna derecha para detallar la rutina seleccionada
- Cabecera con el título de la sección y un subtítulo que indica su usos
- Barra de búsqueda con filtrado en tiempo real por nombre
- Lista de rutinas en forma de tarjetas grid con...
 - Icono de color dinámico según el nombre
 - Nombre de la rutina
 - Días asignados
 - Numero de ejercicios
 - menú de tres puntos para editar
- Panel de detalle a la derecha
 - Cabecera con el nombre de la rutina, días asignados y contador de ejercicios
 - Botones de editar y eliminar
 - Descripción de la rutina
 - Buscador de ejercicios
 - grid de ejercicios con tarjetas mostrando un GIF de dicho ejercicio
 - Tabla de ejercicios añadidos con valores customizables de sets, repeticiones y descansos

Página de Planificación (Planning.jsx)

- Calendario semanal con un grid de 7 tarjetas (de lunes a viernes) con navegación entre semanas
- Tarjetas de cada día que muestra
 - Nombre del día abreviado y su numero
 - Hasta 3 rutinas asignadas con icono y nombre
 - Si no hay una rutina asignada, se mostrara con “Rest Day”
 - Si se arrastra una rutina encima, el borde cambia indicando zona de drop
 - La tarjeta del día actual se resalta visualmente
 - Al hacer clic en un día con rutinas, se expande un panel inferior mostrando el detalle
- Detalle expandido
 - Titulo del día con numero de rutinas
 - Cada rutina como bloque expandible con nombre, descripción y tabla de ejercicios
- Lista de rutinas
 - Todas las rutinas con su icono de color, nombre u descripción (en el caso de que tenga una) y numero de ejercicios
 - Badges de días asignados
 - Drag and Drop para cada rutina, haciendo posible que con arrastrarla y soltarla en un día del calendario, esta se asigne automáticamente

Página de Ejercicios (Exercises.jsx)

- Cabecera con un título indicando la sección actual y un subtítulo indicando la cantidad de ejercicios
- Barra de filtros
 - Buscador de texto
 - Select desplegable para filtra por grupo muscular
 - Botón para resetear los filtros
- Grid de ejercicios con un layout responsive que muestra tarjetas de ejercicios
- Cada tarjeta tiene
 - Imagen GIF del ejercicio, en el caso de no tener se mostrara el logo de la pagina
 - Nombre del ejercicio
 - Tags de partes del cuerpo
 - Etiqueta de músculos objetivo
 - Al hacer click, se mostrara un modal con los detalles del ejercicio
- Paginación con botones para la pagina anterior y siguiente

Modal de perfil (ProfileModal.jsx)

- Renderizado mediante createPortal a document.body para evitar el clipping del backdrop-filter del Sidebar
- Fondo semitransparente oscuro
- Modal centrado con ancho máximo de 440px, fondo de tema actual, border redondeados
- Campos del formulario
 - Avatar circular con la foto de perfil o inicial del nombre
 - Nombre completo
 - Biografía
 - Fecha de nacimiento
 - Numero de teléfono
 - Botón de guardado que cerrara el modal independientemente de si fue exitoso o no

Tema Claro/Obscuro

- Implementado con CSS custom properties en variables.css
- :root define los colores del tema oscuro
- light-theme sobrescribe las variables con colores claros
- El Sidebar tiene un botón para alternar la clase en document.body y persiste en localStorage
- El fondo de la pagina usa un degradado que también cambia con las variables linear-gradient
- El Sidebar tiene un sutil degradado naranja de fondo

Diseño Responsive

La aplicación se adapta a tres tamaños mediante media queries en Routines.css

- 1024px: la columna derecha del layout de rutinas pasa debajo de la izquierda
- 768px: Sidebar se contrae, márgenes se reducen, fuentes se ajustan
- 480px: los controles numéricos se simplifican, las tablas de ejercicios muestran menos columnas

Desarrollo e Implementación (Backend)

Configuración de la aplicación (Program)

El punto de entrada de la aplicación configura todos los servicios necesarios. Se utiliza el patrón de construcción fluida de ASP.NET Core para registrar cada componente

DbContext con PostgreSQL: Se configura el contexto de base de datos usando Entity Framework Core con el proveedor Npgsql. La cadena de conexión se lee desde el archivo appsettings.json o desde la variable de entorno ConnectionStrings_default cuando se ejecuta en Docker

ASP.NET Core Identity: Se añade el sistema de autenticación de Identity con soporte para JWT. Se configuran los requisitos de contraseña y se requiere un Email único

JWT Authentication: Se configura la autenticación mediante tokens JWT. El token incluye 3 promesas, NameIdentifier, Email y Name. Se valida el issuer, audience, lifetime y signing key. La clave de firma se obtiene del archivo de configuración o variable de entorno

CORS: Se configura una política dinámica que lee los orígenes permitidos desde la variable de entorno CORS_ORIGINS. En desarrollo se permiten los orígenes locales, en producción permite el dominio real <https://alejandrolago.com>

Servicios: Se registran los servicios personalizados mediante el patrón de inyección de dependencias. ExerciseApiService se registra con AddHttpClient para gestionar automáticamente el ciclo de vida de HttpClient. ExerciseRepository se registra como Scoped

Autenticación JWT (AuthController)

El controlador de autenticación utiliza UserManager<User> y SignInManager<User> de ASP.NET Core Identity para gestionar el registro y el inicio de sesión

Registro: Al recibir una solicitud POST a /api/auth/register, se divide el nombre completo en nombre y apellidos, se crea una nueva instancia de User con los datos proporcionados, y se utiliza UserManager.CreateAsync para guardar el usuario con la contraseña hasheada. Si el registro es exitoso, se genera un token JWT y se devuelve en la respuesta junto con los datos del usuario. Si falla, se devuelven los errores de validación

Login: Al recibir una solicitud POST a /api/auth/login, se utiliza SignInManager.PasswordSignInAsync para verificar las credenciales. Si son correctas, se obtiene el usuario mediante UserManager.FindByEmailAsync y se genera un nuevo token JWT. Si falla, se devuelve un error 401 con un mensaje genérico

Generación del token JWT: El token se construye con los claims del usuario, una fecha de expiración configurable y se firma con HMAC-SHA256 usando la clave secreta de la configuración

CRUD de Rutinas (RoutineController)

El controlador de rutinas implementa las operaciones CRUD completas protegidas con el atributo [Authorize]. Cada método obtiene el usuario autenticado mediante `userManager.GetUserAsync(User)`.

GET /api/routines: Obtiene todas las rutinas del usuario autenticado incluyendo las relaciones con días de la semana y ejercicios mediante Include y ThenInclude de Entity Framework. Los resultados se mapean a DTOs antes de ser devueltos.

POST /api/routines: Crea una nueva rutina con los datos proporcionados. Los ejercicios se resuelven mediante el método `ResolveExerciseId`, que busca primero el ejercicio en la base de datos local. Si no existe, lo consulta a la API externa mediante `ExerciseApiService.GetExerciseByIdAsync` y lo guarda para futuras consultas.

PUT /api/routines/{id}: Actualiza una rutina existente. Elimina las relaciones anteriores de `RoutineDays` y `RoutineExercises` y las reemplaza con los nuevos datos. Esto se hace con `RemoveRange` y añadiendo los nuevos elementos

DELETE /api/routines/{id}: Elimina una rutina. Las relaciones en cascada (`RoutineDays` y `RoutineExercises`) se eliminan automáticamente gracias al comportamiento `Cascade` configurado en el `DbContext`

Mapeo a DTOs: El método `MapToResponse` convierte una entidad `Routine` en un `RoutineResponse`, incluyendo los nombres de los días de la semana, los ordenes de los días y la lista de ejercicios con su configuración

Gestión del perfil (ProfileController)

El controlador del perfil permite obtener y actualizar los datos del usuario autenticado.

GET /api/profile: Devuelve los datos del perfil del usuario autenticado mapeados a UserProfileResponse. Incluye Email, FullName, FirstName, PhoneNumber, Bio, ProfilePictureUrl y DateOfBirth

PUT /api/profile: Actualiza los campos proporcionados. El nombre completo se divide en nombre y apellidos automáticamente. La fecha de nacimiento se envía desde el frontend como string y se convierte a DateTimeKind.Utc mediante DateTime.SpecifyKind para cumplir con los requisitos de Npgsql para columnas timestamp with time zone.

Integración con API externa (ExerciseApiService)

El servicio ExerciseApiService se encarga de obtener los ejercicios desde la API externa ExerciseDB y cachearlos en memoria para evitar repeticiones.

Carga inicial: La primera vez que se solicita, se realizan 8 peticiones GET a la API externa, una por cada grupo muscular: chest, back, shoulders, upper arms, upper legs, lower legs, waist, cardio. Cada petición usa pageSize=30 para obtener un total aproximado de 240 ejercicios. Entre cada petición hay un retardo de 300ms para evitar saturar la API

Caché en memoria: Los resultados se almacenan en una variable estática cachedExercises de tipo List<ExerciseItemDTO>. Para garantizar la seguridad en entornos concurrentes, se utiliza un SemaphoreSlim que permite que solo un hilo llene la cache mientras los demás esperan.

Búsqueda local: Una vez cacheados los ejercicios, las búsquedas por nombre o filtro de parte del cuerpo se realizan completamente en memoria sobre la lista cacheada, sin necesidad de volver a consultar la API externa

Mecanismo de fallback: Si la API externa no está disponible, el sistema carga los archivos desde un archivo JSON local ubicado en Data/exercises.json. Este archivo contiene datos de ejercicios en formato libre convertidos al mismo DTO mediante una función de mapeo que traduce los nombres de músculos a partes del cuerpo.

Configuración de Entity Framework (AppDbContext)

El contexto de base de datos extiende IdentityDbContext<User> para integrar las tablas de ASP.NET Core Identity con las tablas personalizadas del proyecto

Entidades: Se definen DbSet para Exercises, DayOfWeeks, Routines, RoutineDays y RoutineExercises

Configuración de entidades:

- Exercises: Tiene un índice único en ExternalApiId para evitar duplicados de la API externa
- DayOfWeeks: tiene datos semilla precargados con nombre y orden
- Routines: CreatedAt tiene un valor por defecto now() mediante SQL
- RoutineDays: tiene un índice único compuesto en RoutineId y DayOfWeekId y relaciones con Cascade para Routine y Restrict para DayOfWeek
- RoutineExercises: tiene valores por defecto para Sets(1), Reps(10), RestTimeSeconds(60)